

Lcpy: General documentation

Author: Spyridon Gkousis

Conceptual description of the package

Lcpy is an open-source python package for flexible, parametric, and modular dynamic Life Cycle Assessment (LCA), Techno-economic Analysis (TEA), and Life Cycle Costing (LCC). The investigated system in lcpy is considered as a process (the main process) that consists of other sub-processes. Each sub-process is characterized by its inflows (e.g., materials, energy, water, infrastructure, transport) and its outflows (e.g., products, waste, co-products). The inflows and outflows of the sub-processes are considered as separate sub-sub-processes. The sub-processes consume sub-sub-processes and provide to the main process which models the investigated system (Figure 1).

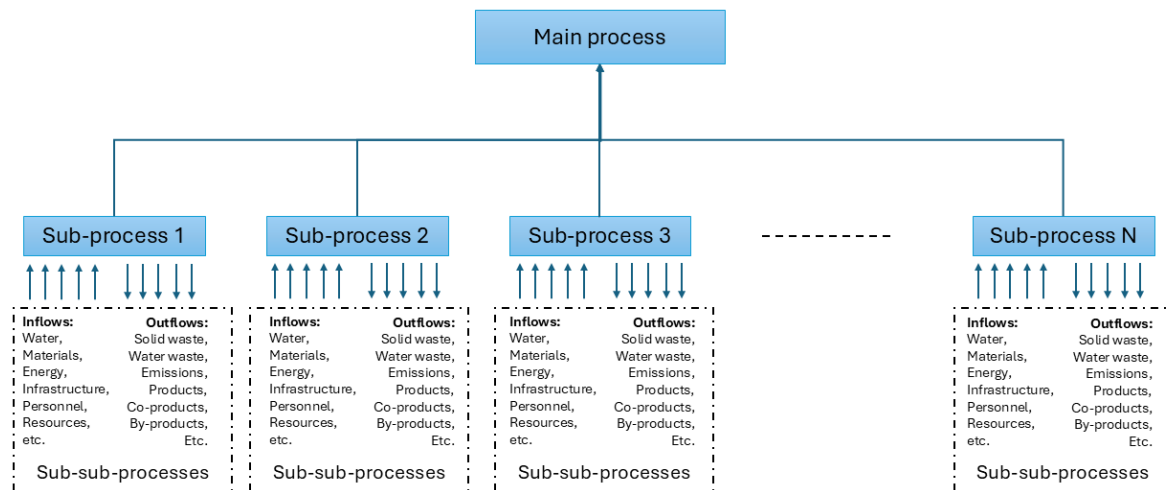


Figure 1: Modelling principle for the main process.

For example, in a chemical manufacturing plant the main process is the chemical production, sub-processes include the mixing, stirring, heating, distillation and other steps and the sub-sub-process refer to, inter-alia, the electricity and solvents consumed in each step. In reality, each sub-process produces an intermediate product that feeds the next sub-process. These processes can be modelled in lcpy by considering the intermediate products of each step as inflows and outflows. Alternatively, the sub-processes can be considered to provide a service to the main process (e.g., the stirring of 1 kg of mixture). In both cases the mass energy balances should be respected.

In a second example, the main process can refer to heat production from a geothermal heat pump, with the sub-processes referring to the electricity consumption, the heat pump manufacturing & installation, and the secondary fluid production (if any).

Lcpy considers that each sub-sub-process is associated with an environmental burden (positive or negative) and a cost. The calculation logic then follows a simple approach where first the unit impacts and costs of each sub-sub-process are considered (e.g., how much Global Warming Impact and Cost to produce 1kWh of electricity. These are then scaled with the amount of each sub-sub-process required for the sub-process to take place (e.g., how much electricity to stir 1 kg of mixture), calculating thus the unit impact and cost of each sub-process. Finally, the unit impacts of each sub-process are scaled

with the amounts required for the main process to take place (e.g., 3 kg of mixture need to be stirred to produce 1 kg of chemical at the end).

Lcpy allows flexible modelling of the sub-sub-processes and sub-processes, with the user being allowed to model absolute amounts or per functional unit, consider multiple scenarios, and introduce a time step for dynamic analysis. The underlying equations are

$$SP_{unit-impact,i,j} = \sum_0^{SSP} (SSP_{unit-impact,i,j} * SSP_{exchange-amount,i,j})$$

$$MP_{unit-impact,i,j} = \sum_0^{SP} (SP_{unit-impact,i,j} * SP_{exchange-amount,i,j})$$

$$SP_{unit-cost,i,j} = \sum_0^{SSP} (SSP_{unit-cost,i,j} * SSP_{exchange-amount,i,j})$$

$$MP_{unit-cost,i,j} = \sum_0^{SP} (SP_{unit-cost,i,j} * SP_{exchange-amount,i,j})$$

Where i,j are the scenarios and time steps modelled, and MP, SP, and SSP are the main process, sub-process, and sub-sub-processes respectively.

Lcpy allows a modular approach where the calculated main process can be used as a sub-process to another main process, allowing for flexible modelling of large systems (Figure 2).

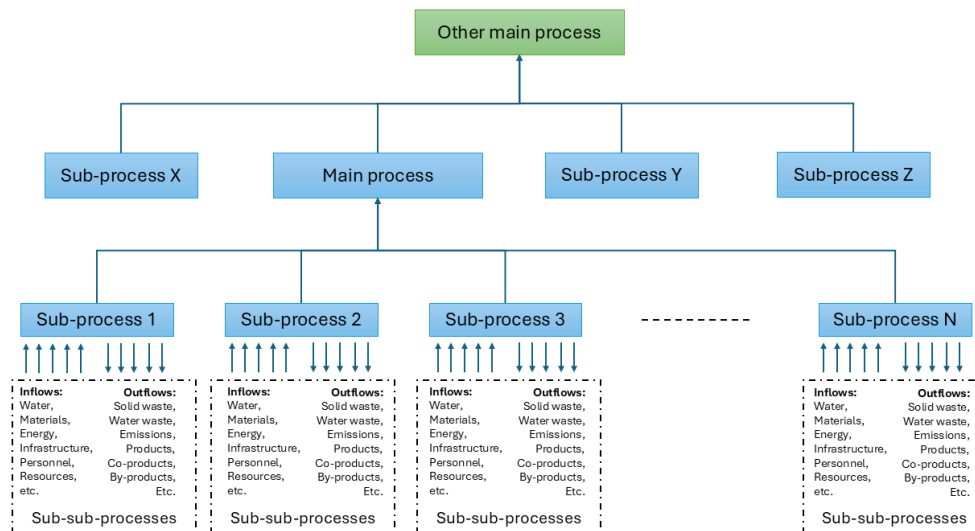


Figure 2: Modelling principle to use a main process as a sub-process to another main process

Lcpy prompts its user to enter in structured dictionaries and lists

- 1) The names of sub-sub-processes, sub-processes, and main process
- 2) Ecoinvent keys for each sub-sub-process to calculate its unit environmental impact (alternatively this can be entered directly). (For the LCA analysis)
- 3) The amounts of each sub-sub-process needed in the sub-process and the amounts of each sub-process needed for the main process.
- 4) The costs/revenues associated with the above amounts (for the economic analysis).
- 5) A few other parameters needed for the modelling (e.g., number of scenarios, discount rate etc.)

Lcpy then provides functions to perform simple LCA and LCC, include uncertainty (Scenarios, Monte Carlo) or dynamics (dynamic LCA and dynamic impact assessment), and perform Global Sensitivity Analysis and optimization.

The user is let to fill the structured dictionaries and list (data 1-5 above) either as single scalars, numpy arrays coming from a simulation model, or as a parametric model to allow for the above-mentioned flexible analysis.

Prerequisites:

Lcpy uses numpy and cython for its calculations, pandas for results handling and storing, and matplotlib and seaborn for visualization. Any other library required for the parametric modelling or to perform GSA or optimization can be separately installed.

Lcpy uses brightway2 (bw2) to calculate unit impacts, inventories, and characterized inventories for the sub-sub-processes and for several impact categories. However, the bw environment must be separately set up to not interfere with the lcpy environment. The user needs to create a bw2 environment, create a bw project with an ecoinvent copy in it (optional), and pass the path to the bw2 environment, along with the name of the bw2 project, and the used bw database names in a structured dictionary (brightway_configuration_dictionary) in the lcpy. Lcpy will then communicate efficiently with the bw2 environment to calculate the required information.

Definition of general parameters:

Before defining the parametric model and running the calculations some general parameters need to be defined. These include a list with the impact assessment methods, and a list with their units and names (for the LCA). These can be derived directly from the bw2 environment or via the Activity Browser interface (it is generally recommended to use AB to derive the LCA-related information needed, especially for LCA-beginners). The number of scenarios to be investigated, the total operation timeframe considered, the time step, the number of how many sub-processes concern construction activities, and the number of time steps required for the construction are also defined. Note that not all these parameters are needed for all types of analysis and

depending on the use case they can be skipped. A directory in which to store the results also needs to be set up.

Parametric models and definition of sub-processes and main process:

For the LCA modelling each sub-process is defined with a dictionary and a list, while for the economic modelling each sub-process is defined with two dictionaries and two lists.

- **LCA:**

The dictionary for each sub-process contains #sub-sub-processes elements. The keys are the names of the sub-sub-processes and the values are ecoinvent keys that will be used to calculate their unit impacts (it is recommended to look these up using the AB or advise the bw2 tutorials on how to derive them).

The list for each sub-process contains also #sub-sub-processes elements, which will be matched element-wise with the dictionary entries (so please ensure that the entry sequence is same). The list contains scalar values or numpy arrays with the amount of exchange needed for each sub-sub-process by the sub-process. This can be modelled with a single value or in time and can be the result of a parametric or simulation model which was run before starting the definition of the sub-processes.

The main process is also modelled with a dictionary and a list following the same logic as above, though in this case the sub-processes replace the sub-sub-processes. In this case however, the values of the dictionary can remain empty strings since the unit impact of each sub-process will be calculated previously.

- **Economic analysis:**

Two pairs of dictionary-list are created for each sub-process for the economic analysis. The first pair concerns the cost generating activities and the second the revenue generating activities.

For the first pair, the dictionary for each sub-process contains #sub-sub-processes that generate cost elements. The keys are the names of the sub-sub-processes and the values are empty strings. The list contains also that generate cost #sub-sub-processes elements, which will be matched element-wise with the dictionary entries and contains scalar values or numpy arrays with the amount of exchange needed for each sub-sub-process by the sub-process multiplied by the respective cost price.

For the second pair, the dictionary for each sub-process contains #sub-sub-processes that generate revenue elements. The keys are the names of the sub-sub-processes and the values are empty strings. The list contains also #sub-sub-processes that generate revenue elements, which will be matched element-wise with the dictionary entries and contains scalar values or numpy arrays with the amount of exchange needed for each sub-sub-process by the sub-process multiplied by the respective cost.

The main process is modelled with one dictionary and one list with the sub-processes replacing the sub-sub-processes.

Restructuring:

Before the calculations start the dictionaries modelling each sub-process need to be wrapped in a list as well as the lists modelling the exchange amounts. The sequence should be the same in the two lists and ideally should follow the sequence of defining the sub-processes.

For the economic analysis different lists should be constructed for the cost- and revenue-generating activities (so in total four lists). In case construction activities are modelled separately than operational activities (dynamic modelling) then the former should be put first.

Mapping dictionaries are then created mapping sub-processes names to sub-sub-processes and sub-processes names to the respective exchanges. The `create_mapping` function can be used for this. Also, the `create_list_with_unique_activities` can be used to create a list with all the unique sub-sub-processes occurring.

For the economic analysis mapping dictionaries should be created for the sub-processes causing operational costs, construction costs, and all of them together. Also, mapping dictionaries should be created for the sub-processes causing operational revenues, construction revenues, and all of them together. The unique sub-sub-processes names list should be created for the cost and revenue causing sub-sub-processes (two lists).

Calculations:

- **Simple LCA**

A dictionary mapping the name of the main process to the exchange amounts is created as well as a dictionary mapping the name of the main process to the names of the sub-processes (this can be created easily as shown in the examples and a function is not provided).

An `mpLCAer` instance is created. The `brightway2` environment is loaded (function : `import_isolated_environment`), and the unit impacts and inventories for each sub-sub-process are derived (function: `lca_calculations`).

An `ExchangeHolder` and an `ImpactCalculator` instances are created for handling the sub-processes and their functions `create_exchange_arrays` and `impact_calculation_simple` are used to calculate the unit impact of each sub-process.

An `ImpactHandler` instance is created for the sub-processes and the `create_name_dictionaries` function is run that creates a dictionary mapping the sub-process names to sub-sub-processes names. The `create_dataframes` function of the

impact handler function is run which to create dictionaries with dataframes holding the unit_impacts and the contribution analysis of each sub-process.

An ImpactHandler instance is also created for the main process and a list holding the name of the main process. The calculate_total_unit_impact is run that creates a dictionary with the unit impacts of each sub-process. An ExchangeHolder and an ImpactCalculator instances are created for handling the main process as well.

The create_exchange_arrays and impact_calculation_simple of the ExchangeHolder and an ImpactCalculator instances for the main process are used to calculate the unit impact of the main process (the final result thus). The create_dataframes function of the impact handler function is run which to create dictionaries with dataframes holding the unit_impacts and the contribution analysis for the main process.

Storing and visualization

A list with the dataframes holding the contribution analysis results for the sub-processes and the main process as well as a list with the names of excel sheets to store the results need to be created. A dictionary with these dataframes for each sub-process and the main process is created (create_dataframe_dict). The same procedure is repeated for the absolute impact results.

File paths for storing the results are created and the save_dataframes_to_excel function is run to save the results stored in the dictionaries.

The contribution analysis per sub-sub-process can be calculated with the contribution_to_total_impact_per_sub_sub_processes function of the ImpactHandler instance for the sub-processes.

The LCA results can be visualized with the plot_stacked_percentage_bar_grid/plot_stacked_percentage_bar_sub_processes functions that plot the contribution analysis results per sub-sub-process and the plot_stacked_percentage_barchart_seaborn function that plots the contribution analysis for the main process and for each sub-process separately,

In case a contribution analysis is needed for a sub-sub-process separately the relevant brightway functions can be used. The mpLCAer class includes these functions for bw2, taken from the github bw tutorials and includes them in the contribution_analysis_in_technosphere and contribution_analysis_in_biosphere functions.

Using the main process as a sub-process in a new main process

The above analysis calculates the unit impact for a main process considering that it basically consists of ecoinvent activities (sub-sub-process). To re-use these results as a sub-process in another main process without needing to re-run the calculations and re-

do the modelling the following need to be done (similar for other externally calculated sub-processes)

First, the new main process and its sub-processes that consist of ecoinvent activities are modelled as usual. A dictionary and a list for the external sub-process can also be constructed to keep the information available. The LCA is ran normally until the `lca_calculations` is run. Next, some appends to the created lists/dataframes/dictionaries to include the externally calculated sub-process are done (see the relevant example)

- 1) The `unit_impacts` dictionary is appended with the `unit_impacts` of the external sub-process.
- 2) The names mapping dictionary is appended with the name of the external sub-process.
- 3) The exchanges mapping dictionary is appended with the exchanges of the external sub-process.
- 4) The list that holds the names mapping dictionaries for each sub-process is appended with the dictionary for the external sub-process.
- 5) The list that holds the exchanges mapping dictionaries for each sub-process is appended with the dictionary for the external sub-process.
- 6) The dictionary and list modelling the new main process is appended with the relevant information for the external sub-process.

All next modelling is the same as for the simple LCA.

The above calculations concern only the impact results and not the inventory results or the impact caused by environmental flow. Functions are provided for extracting these results as well (see example), which are explained in a next section explaining the dynamic LCA.

- **Monte Carlo (2-D) LCA**

The modelling process for the Monte Carlo (MC) LCA is identical with that described for the simple LCA until the `mpLCAer` instance is created, the `brightway2` environment is loaded (function : `import_isolated_environment`), and the unit impacts and inventories are derived (function: `lca_calculations`).

The critical difference with the simple LCA is in the parametric modelling. In this case the exchange values (held in the lists that model each sub-process and the main process) are not scalar values but numpy arrays of dimensions (scenarios, 1).

To handle this a `fast_calculator` instance is created. The unit impacts of each sub-sub-process is scaled with the exchange amounts required for each sub-process in each MC scenario calculating thus the unit impact of each sub-process in each scenario (`fast_calculator.calculation_static_lcia` function)

n). Next, the unit impacts of each sub-process are scaled for each scenario with the respective exchange amount needed for the main process, again for each scenario (`fast_calculator.calculation_impact_senarios` function). The results which are stored in a dictionary attribute of the `fast_calculator` instance can be put into a dataframe and stored with the `store_scenario_results` function which as well creates boxplots and calculates some basic statistics.

- **Dynamic LCA**

The modelling process for dynamic LCA is identical with that described for the simple LCA until the `mpLCAer` instance is created, the `brightway2` environment is loaded (function : `import_isolated_environment`), and the unit impacts and inventories are derived (function: `lca_calculations`).

The critical difference with the simple LCA is in the parametric modelling. In this case the exchange values (held in the lists that model each sub-process and the main process) are not scalar values but numpy arrays of dimensions (scenarios, time steps), modelling the amount of exchange needed at each time step, which can be the result of a simulation or parametric/projection model. In addition, in the dynamic LCA the construction sub-processes are modelled separately than the operational sub-processes. The numpy arrays for the former are of dimensions (scenarios, time steps needed for the construction) and for the latter of dimensions (scenarios, operational time steps). Nevertheless, `lcpy` is flexible to allow the user to consider or not the construction.

A `fast_calculator` instance is again created to handle the scaling of the unit impacts and inventories with the exchange amounts. The unit impacts of each sub-sub-process is scaled with the exchange amounts required for each sub-process in each scenario and time step calculating thus the unit impact of each sub-process in each scenario and time step (`fast_calculator.calculation_static_lcia` function). Next, the unit impacts of the sub-processes are scaled with the exchange amounts required for the main process for each scenario and time step (`fast_calculator.impact_calculation_total` function). This function handles differently the construction and operational sub-processes, thus the number of construction processes and construction years need to be passed in. It calculates the unit impact of the main process at each time step and for each scenario.

Now the impacts have been calculated, but there is no information about the environmental inventory flows and the impact they cause. To explore this and allow for dynamic impact assessment, some more functions can be run.

First, the `derive_technosphere_and_biosphere_dictionaries` of the `mpLCAer()` instance is run to derive the biosphere and technosphere dictionaries using `bw2`. These dictionaries can be stored with the `store_dictionaries` function. The unit inventories holding the environmental flows of each sub-sub-process in each sub-process are aggregated per environmental flow

(`fast_calculator.sum_inventory_of_sub_sub_process_per_sub_process` function), and the same is done for the characterized unit inventory which holds the impact cause by each environmental flow (`fast_calculator.sum_characterized_inventory_of_sub_sub_process` function). Once the unit inventories and unit characterized inventories have been calculated for each sub-sub-process and stored in the relevant attributes they have to be scaled with the exchange amounts needed for each sub-process to calculate the unit (characterized) inventory of each sub-process and then scale this with the exchange amounts required for the main process to calculate the unit (characterized) inventory of the main process. The logic for this scaling is the same as that performed for the unit impacts and is done with four functions of the `fast_calculator` instance

```
(emissions_calculation_in_time,      characterized_inventory_calculation_in_time,  
emissions_calculation_in_time_total,  
characterized_inventory_calculation_in_time_total)
```

These results can be stored in dataframes and passed in a dictionary and inspected with the `make_characterized_inventory_dfs` function.

The core results have been calculated now. The impact caused across the time steps and for each scenario can be summed (`fast_calculator.calculate_total_impact_per_category_and_scenario`). If the result is not calculated per functional unit and the result per functional unit is sought two functions can be run:

```
(fast_calculator.calculate_total_fu_per_scenario,  
fast_calculator.per_fu_per_category_per_scenario)
```

The evolution in time of the absolute impact and of the per functional unit impact can be calculated with:

```
(fast_calculator.calculation_of_the_in_time_impact_evolution_absolute,  
fast_calculator.cum_calculation_of_the_in_time_impact_evolution_per_fu_actual)
```

Storing and visualization

The calculated results can be stored and visualized with several functions provided by `lcpy`:

```
create_scenario_dataframes  
save_scenario_dataframes  
create_and_save_dataframes_per_scenario_and_per_category  
create_and_save_dataframes_time_series  
store_scenario_results  
visualize_dict_lineplots
```

plot_bar_charts_by_category
visualize_stacked_bar chart_time_series_impact

Using the main process as a sub-process in a new main process

First, the new main process and its sub-processes that consist of ecoinvent activities are modelled as usual. A dictionary and a list for the external sub-process can also be constructed to keep the information available. The LCA is ran normally until the lca_calculations is run. Next, some appends to the created lists/dataframes/dictionaries to include the externally calculated sub-process are done (see the relevant example). The externally calculated construction sub-processes need to be appended with the construction sub-processes at the start of the dictionaries and lists, while the externally calculated construction sub-processes need to be appended with the construction sub-processes at the start of the dictionaries, and lists. For this OrderedDict objects are used.

- 1) The unit_impacts dictionary is transformed to an ordered dictionary and the unit impact of the external construction sub-processes are appended at its beginning, while the unit impact of the external construction sub-processes are appended at its end.
- 2) We do the same appends following the same logic for the impact_per_process dictionaries.
- 3) The names mapping dictionary is appended with the names of the external sub-processes. The construction sub-processes are appended at the beginning and the operational at the end.
- 4) The exchanges mapping dictionary is appended with the exchanges of the external sub-process. The construction sub-processes are appended at the beginning and the operational at the end.
- 5) The list that holds the names mapping dictionaries for each sub-process is appended with the dictionary for the external sub-process.
- 6) The list that holds the exchanges mapping dictionaries for each sub-process is appended with the dictionary for the external sub-process.
- 7) The dictionary and list modelling the new main process is appended with the relevant information for the external sub-process. The construction sub-processes are appended at the beginning and the operational at the end.

All next modelling is the same with what described above.

- *Dynamic Impact assessment:*

The above analysis performs a “semi-dynamic” LCA using a dynamic LCI but static characterization factors. Lcpy allows to load dynamic characterization factors and conduct dynamic impact assessment using these.

The dynamic characterization factors are to be input in an excel file and loaded in the numpy arrays. The way of performing dynamic impact characterization follows the paper of (Gkousis et al. 2022) with Fixed time horizon Characterization factors (FCFs) and instantaneous Characterization Factors (ICFs) inspired by Levasseur et al. 2010.

Instructions to construct the excel files with dynamic characterization factors:

First, the `store_dictionaries` function is ran with the value held in the `biosphere_dict` dictionary as input. This creates an excel file that has all biosphere flows. The excel has in the first column the keys and in the second column the names of the biosphere columns. Next, this excel should be modified to

- 1) Include in the first row starting from the third row the number of years for which dynamic CFs will be filled with (e.g., 1-100).
 - 2) Copy the excel sheet and create multiple sheets for modelling the various cases of characterization factors (impact categories covered and FCFs/ICFs), name the sheets accordingly (e.g., GWP-FCF, GWP-ICF).
 - 3) Find the biosphere flows for which dynamic CFs are to be used and fill the rows with the values of the CFs evolving in time. FCFs are filled for `construction_years + time_steps` years, while ICFs are filled for TH years. Note: dynamic CFs are filled ONLY for the biosphere flows that they are available. All other cells are left empty!!!
 - 4) Once the excel file, `***dyn_excel_file.xlsx***` is created, it can be fed into two functions `create_fcf_array` and `create_icf_array` to create the full Characterization Factor matrix.
-

The impact with dynamic impact assessment methods can be calculated using FCFs or ICFs with two functions of the `fast_calculator` instance (`calculation_dynamic_LCIA_fcf`, `calculation_dynamic_LCIA_icf`), which works on the inventory calculated in the previous steps.

Next, the same calculations with the semi-dynamic LCA are run, though the functions are named differently and receive the name of the impact category which is assessed with dynamic CFs (only impact category one at the time can be examined). The functions are re-written to handle this and are named:

```
calculate_total_impact_per_category_and_scenario_fcf
per_fu_per_category_per_scenario_fcf
calculation_of_the_in_time_impact_evolution_absolute_fcf
cum_calculation_of_the_in_time_impact_evolution_per_fu_actual_fcf
```

For the visualization and storing of the results the same functions can be used:

```
create_scenario_dataframes  
save_scenario_dataframes  
create_and_save_dataframes_per_scenario_and_per_category  
create_and_save_dataframes_time_series  
store_scenario_results  
visualize_dict_lineplots  
plot_bar_charts_by_category  
visualize_stacked_barchart_time_series_impact
```

Additional functions are provided to match the results to the environmental flows and plot the impact evolution of the impact caused by each environmental flow

```
map_fcf_icf_results_to_flows  
plot_fcf_line_per_flow  
plot_icf_line_per_flow
```

- **LCC and TEA**

A dictionary mapping the name of the main process to the exchange amounts is created as well as a dictionary mapping the name of the main process to the names of the sub-processes (this can be created easily as shown in the examples and a function is not provided).

A FastTeaCalculator instance is created. For its creation the number of construction sub-processes, the number of construction years, the number of scenarios investigated, and the discount rate have to be provided.

Two functions are run that calculate the costs and revenues occurring during the operation discounted at the first operational year (FastTeaCalculator.cost_calculation, FastTeaCalculator.revenue_calculation).

The contribution analysis for the costs and the revenues of each sub-process can be calculated before making any discounting with the calculate_undiscounted_cost_contributions_per_process function.

The total cost and revenue for the main process and its evolution in time cumulatively and non-cumulatively can be calculated (discounted at the first operational year or the first construction year) with the total_cost_calculation, total_revenue_calculation, and total_cost_calculation_including_infrastructure functions.

Then, the refine_revenue_dicts function has to be run to restructure the revenue dictionaries for later use.

The cost and revenues have been calculated and can be stored and visualized using the below functions:

create_and_save_dataframes_time_series
visualize_dataframes_time_series
visualize_npv_in_time
visualize_stacked_barcharts

The visualize_npv_in_time function simply visualizes the NPV, the NPV can be calculated and stored with the npv_calculation function.

Functions to calculate the lcoe and its development over time, the annualized cost, and the minimum selling price are also provided.

FastTeaCalculator.lcoe_calculation

FastTeaCalculator.annualized_cost_calculation

FastTeaCalculator.msp_calculation

Multiple results can be stored and visualized with the npv_results_storage_and_vis, the create_and_save_dataframes_time_series and visualize_dataframes_time_series can be used again to store and visualize the results. Also, the create_dictionary_with_dfs creates dictionaries with dataframes holding the costs and revenues which can be used to calculate the contribution of each sub-sub-process to the total cost or revenue of the main process.

Using the main process as a sub-process in a new main process

The calculated main process above (or any externally calculated main process) can be used as a sub-process in another main process calculation. The previously main process can be modelled directly with two dictionaries and two lists representing the cost- and revenue-generating activities. Since the analysis for the latter has been performed, only one sub-sub-process for the costs and one for the revenues can be used and a function of the FastTeaCalculator instance that sums these costs and revenues as modelled previously can be used (sum_sub_process). Alternatively, the costs/revenues can be input manually as before. Note that the construction and operational sub-processes of the previous main process have to be considered separately in the modelling of the new main process.

lcpy.hvs.hvs

- **create_name_dictionaries**

This function takes a dictionary and a list of dictionaries and returns a new dictionary that

maps each key from the dictionary to the list of keys from the corresponding dictionary in the list.

- ***create_dataframe_dict***

This function takes a list of pandas DataFrames and a parallel list of names, verifies they're the same length, and returns a dictionary mapping each name to its corresponding DataFrame.

- ***save_dataframes_to_excel***

This function takes a dictionary of pandas DataFrames and writes each one to its own sheet in a single Excel workbook at the specified file_path.

- ***plot_stacked_percentage_bar_sub_processes***

This function takes a dictionary of DataFrames keyed by category, selects the top top_x entries (others grouped as "Other") for a given key, converts their values to percentages, and produces a horizontal stacked bar chart saved to filepath.

- ***plot_stacked_percentage_bar_grid***

Does the same as *plot_stacked_percentage_bar_sub_processes* but plots the results in a grid of 3x3

- ***plot_stacked_percentage_barchart_seaborn***

This function takes a pandas DataFrame whose rows are the sub-processes and columns are the impact categories, converts each column to percentages (so they sum to 100%), and draws a stacked bar chart using Seaborn's palette. It saves the figure to filepath and can optionally display it.

- ***store_scenario_results***

This method takes a pandas DataFrame with the results of the scenarios, an output directory path, a list of unit labels matching each column, figure size, DPI, and an optional suffix. It makes sure the directory exists, saves df to scenarios_dataframe.xlsx, writes df.describe() to scenarios_describe_{name}.xlsx, then loops over each column to draw, label, and save a boxplot PNG (boxplot_{column}_{name}.png) using the corresponding unit. It returns None (all results are written to disk).

- ***make_characterized_inventory_dfs***

This function takes a 4D NumPy array, a list of names, a dictionary mapping flow identifiers to labels, and a scenario_index. It first checks that the lengths match. It then slices out the 2D array for the given scenario and, for each category, builds a pandas

DataFrame with flow labels as the index and time-step integers as the columns. Finally, it returns a dictionary mapping each category name to its corresponding DataFrame.

- ***create_scenario_dataframes***

This function takes a NumPy array x (shape (categories, scenarios, years) or 2D which it pads to 3D), a list of impact category names, and a list of scenario identifiers. It verifies that the dimensions match, then for each scenario it slices out the (categories $\times$ years) matrix, transposes it to (years $\times$ categories), wraps it in a pandas DataFrame (columns = impact categories, rows = years), and returns a dict mapping each scenario name to its DataFrame.

- ***save_scenario_dataframes***

Given a dictionary of DataFrames, a list of names to include, and file naming parameters, this function writes only the selected DataFrames into separate sheets of an Excel workbook named impact_{name}_{per_fu}_{infr_op}.xlsx in output_dir, and returns None (with the file written as a side-effect).

- ***create_and_save_dataframes_per_scenario_and_per_category***

This function takes a 2D array, a list of column names, a file path (file_path), and a boolean save flag; it constructs a pandas DataFrame from the array with those column names, optionally writes it to an Excel file at file_path when save=True (otherwise it prints a notice), and returns the DataFrame.

- ***create_and_save_dataframes_time_series***

This function takes a 2D or 3D NumPy array, a list of category names, a list of scenario names, an excel filename (excel_file_name), and a save flag; it converts each slice of the array into a pandas DataFrame with scenarios as columns (one per impact category if 3D, or a single DF if 2D), optionally writes them to excel_file_name (one sheet per category), prints a notice if not saved, and returns the list of DataFrames.

- ***visualize_dict_lineplots***

This function takes a list of dataframes with the impact and visualizes the impact as a line developing in time the selected impact categories and scenarios.

- ***plot_bar_charts_by_category***

This function takes a dictionary of dataframes with the impact for each scenario and visualizes the impact as barcharts developing in the operational time the selected impact categories and scenarios.

- ***visualize_stacked_barchart_time_series_impact***

This function takes the numpy array holding the contributions of each sub-process to the main process in time and plots the stacked barcharts developing in time.

- ***map_fcf_icf_results_to_flows***

This function takes a mapping of flow indices to environmental flows (biosphere_dict), a list of scenario_names, and a 3D NumPy array of shape (flows × scenarios × time), and returns a nested dictionary where each biosphere label maps to a sub-dictionary of scenario names to their corresponding time-series impact arrays.

- ***plot_fcf_line_per_flow***

Takes the dictionary calculated by the *map_fcf_icf_results_to_flows* function when fed with the FCF results, the scenarios for which to plot and the name of the environmental flows and plots the evolution of the impact caused by these flows. The picture is saved at a specified directory.

- ***plot_icf_line_per_flow***

Takes the dictionary calculated by the *map_fcf_icf_results_to_flows* function when fed with the ICF results, the scenarios for which to plot and the name of the environmental flows and plots the evolution of the impact caused by these flows. The picture is saved at a specified directory.

- ***plot_icf_line***

Plots the total impact calculated with the ICFs.

- ***sum_scenarios_per_flow***

Given the dictionary calculated by the *map_fcf_icf_results_to_flows* when fed with the results calculated with the ICFs and a list with names of environmental flows, it sums the impact caused by the environmental flows for each time step and for the scenarios specified.

lcpy.hvs.filling_dyn_cfs

- ***calculate_fcf***

This function takes as input the filepath to the dyn_excel_file.xlsx, the sheet_name from which to read the FCFs for the desired impact category, and the impact_category name (as read by bw2). This function runs a bw2 calculation and derives the static characterization factors. It creates a numpy array that has the static CFs for the environmental flows that dynamic FCFs are not available, and dynamic FCFs for the environmental flows that these are available.

- ***calculate_icf***

This function operates with a very similar way with the *calculate_fcf* but creates a numpy arrays holding the ICFs for a selected category. The only difference is on how the numpy array is saved. For biosphere flows for which ICFs are available on the excel these are kept in the numpy arrays. For the other biosphere flows, the first column holds the static characterization factor for the selected impact assessment method and the last columns holds the value 20598. The in-between values are zero.

lcpy.hvs.map_dicts

- ***create_mapping***

This function takes a dictionary of keys and a list of dictionaries or sequences, optionally slicing off or keeping a leading segment based on *number_of_infrastructural_processes* and the *tea_op/tea_con* flags, and returns a new dictionary mapping each (possibly sliced) key to the corresponding list of values or keys from each element of the list.

- ***create_list_with_unique_activities***

This function takes a list of dictionaries and returns a list of all unique keys found across those dictionaries.

- ***store_dictionaries***

This function takes a dictionary, an output directory, and a file name; it converts the dict's items into a two-column pandas DataFrame, saves that DataFrame as an Excel file at the specified directory, and returns None (all work is done via the file side-effect).

lcpy.hvs.hvs_tea

- ***calculate_undiscounted_cost_contributions_per_process***

Takes dictionaries of operational and infrastructure subprocess names (*names_op*, *names_infr*), corresponding exchange arrays (*exch_op*, *exch_infr*), column headers (*header*), a mapping for sheet names (*total_names_dict*), an output directory (*target_dir*), optional filenames (*file_name_total*, *file_name_contributions*), a revision tag (*rev*), and indices for scenario and year (*scen*, *year*); it extracts the relevant 1D exchange data, uses another internal function to make absolute and percentage DataFrames, and uses visualization and storing internal functions to create barcharts and save the results in and Excel sheets.

- ***visualize_dataframes_time_series***

This function takes a list of time-series DataFrames their corresponding names (*impacts*), an index or list of scenarios to plot, axis labels (*xlabel*, *ylabel*), a plot title, an output filepath prefix, figure size and resolution (*fig_size*, *dpi*), and an offset to skip initial rows. It then iterates over each DataFrame/category pair, plots the specified scenarios (from offset onward) on a single axis with labels and legend, saves each chart as a PNG

(naming it either {filepath}_{scenario}.png if a single scenario or {filepath}_all_scenarios.png otherwise), and closes the figure. All outputs are written to disk; the function returns None.

- ***visualize_npv_in_time***

This function takes two lists of revenue and cost DataFrames (df_rev, df_cost), a list of scenario names (scen_names), an output filepath, and optional figure size and DPI; it computes net present value for each scenario by subtracting costs from revenues, plots all NPV time series on one chart with labels, saves the figure to filepath, and returns None.

- ***visualize_stacked_barcharts***

Takes a mapping of category keys, a 3D array of cost data, scenario names, a step_for_vis, an output excel_file_path, a scen_to_visualize, and axis labels (yaxis_label, xaxis_label, rel); it builds a dict of DataFrames for each scenario from cost_arrays, then calls an internal visualization function on the chosen scenario to generate and save the stacked bar chart.

- ***npv_results_storage_and_vis***

This function takes parallel lists of arrays (list_values) and their corresponding names (list_names), an output directory (target_dir), and an optional Excel filename (file_name). It builds a DataFrame mapping each name to its NPV series, saves it to {target_dir}/{file_name}, writes descriptive statistics to scenarios_describe_costs.xlsx in target_dir, then generates and saves a boxplot PNG (boxplot_{column}.png) for each scenario column. All outputs are written to disk; the function returns None.

- ***create_dictionary_with_dfs***

This function takes a dictionary mapping of processes to their subprocess names (mapping_dict), a dictionary of cost arrays (cost_dict), a scenario_index to select which scenario to analyze, and a name to use as the DataFrame column header. For each process present in both inputs, it: extracts the cost slice for scenario_index and sums over time steps to get a 1D array of total cost per subprocess, computes each subprocess's contribution fraction, builds two pandas DataFrame with the absolute cost/revenues and the contribution analysis results and puts these into dictionaries which are returned.

- ***contribution_per_sub_sub_processes***

Calculates a list with the cost or revenue contribution to the main process per subprocess.

- ***mpLCAer***

Class that can run brightway2 from a remote environment. It takes the number of cpu to use, the impact assessment methods to use, and a dictionary stating the brightway2 environment path, project name, and databases used.

- ***mpLCAer.import_isolated_environment***

Imports the bw2 environment in lcpy

- ***mpLCAer.lca_calculations***

Given a dictionary with values lists of ecoinvent keys and keys sub-process names it creates a three dictionary attributes of the *mpLCAer* instance: *unit_impacts*, *unit_inventory*, *unit_char_inventory* that have as keys the names of the sub-processes and as values numpy arrays that hold the unit impact, inventory and characterized inventory for each sub-sub-process included in the sub-process.

- ***mpLCAer.contribution_analysis_in_technosphere***

Given a list with an ecoinvent key and a list with impact assessment methods it runs a bw2 calculation and returns the a list with the contribution analysis results in the technosphere for the activity represented by the ecoinvent key.

- ***mpLCAer.contribution_analysis_in_biosphere***

Given a list with an ecoinvent key and a list with impact assessment methods it runs a bw2 calculation and returns a list with the contribution analysis results in the biosphere for the activity represented by the ecoinvent key.

- ***mpLCAer.derive_technosphere_and_biosphere_dictionaries***

Creates dictionary attributes of the *mpLCAer* instance that hold the environmental flows and technosphere activities of the ecoinvent database used in the bw2 project.

lcpy.calculators.helpers

- ***ExchangeHolder***

Class that is used only in the simple LCA. It has one function: *create_exchange_arrays* that is used to broadcast the exchange amounts of each sub-process to the number of impact categories explored.

- ***ImpactCalculator***

Class that is used only in the simple LCA. It has one function: *impact_calculation_simple* that is used to scale unit values of impact with the amount of exchanges given the dictionaries holding the unit impacts and the associated exchange amounts (they should have the same length).

- ***ImpactHandler***

Class that is used only in the simple LCA. It takes the list with impact categories names.

- ***create_dataframes***

This method takes a dictionary of impact arrays and a parallel dictionary of subprocess name, checks they align, then for each entry computes the total impacts and percentage contributions, storing the results as pandas DataFrame in *df_impacts* and *df_contributions*, and the raw totals in *total_impact_arrays*.

- ***calculate_total_unit_impact***

This method takes a dictionary of NumPy arrays and a key identifier (name), vertically stacks all the arrays in the dictionary, and stores the resulting 2D array in *total_unit_impact* under that key.

- ***contribution_to_total_impact_per_sub_sub_processes***

This method computes, for each impact category, the total contribution of every sub-sub-process by combining each subprocess's percentage contribution with the overall category impact. It stores the results as pandas DataFrames in *contribution_per_sub_sub_process* (one sheet per category) and can optionally save them all into an Excel workbook.

lcpy.calculators.env_calc

- ***fast_calculator***

A class that runs cython functions to quickly scale environmental impacts with the relevant exchange amounts for multidimensional calculations. It provides a class that communicates the modelling of the user and the bw2 results to cython functions and stores the results in its attributes which are then used to make the results analysis, storage, and visualization

It has several dictionaries attributes:

impact_per_process, impact, total_impact, total_impact_contributions,
total_impact_per_category_and_scenario,
total_impact_per_category_and_scenario_fcf, total_fu, total_fu_per_scenario,
total_impact_per_fu_per_category_per_scenario, total_impact_per_fu,
total_impact_per_fu_per_category_per_scenario_fcf, cum_impact_in_time,
cum_impact_in_time_per_fu, cum_fu, total_impact_infr,
total_impact_contributions_infr, total_impact_op, total_impact_contributions_op,
summed_unit_inventories, summed_unit_characterized_inventories, inventory,
inventory_per_process, characterized_inventory, characterized_inventory_per_process,
total_inventory, total_inventory_per_process, total_inventory_infr,
total_inventory_contributions_infr, total_inventory_op,
total_inventory_contributions_op, total_char_inventory,
total_char_inventory_per_process, total_char_inventory_infr,
total_char_inventory_contributions_infr, total_char_inventory_op,
total_char_inventory_contributions_op, total_impact_fcf_per_flow, total_impact_fcf,
total_impact_icf_per_flow, total_impact_icf, cum_impact_in_time_fcf,
cum_impact_in_time_per_fu_fcf, total_impact_per_fu_fcf,

- ***calculation_static_lcia***

This method zips together the dictionaries holding the exchange amounts and the unit impacts for each sub-sub-process, calls the `calc.impact_calculation_static_lcia`` function on each pair of values which scales the unit impacts with the exchange amounts and stores the resulting per sub-process impacts and total impacts in instance attributes (`impact_per_process`, `impact`).

- ***calculation_impact_senarios***

This method takes the main process name, the dictionaries holding the exchange amounts for the main process, and the unit impacts for each sub-process,, verifies they align in length, then scales the unit impacts with the exchange amounts for each scenario via the `calc.impact_calculation_only_senarios`, and stores the results in instance attributes (`total_impact_contributions`, `total_impact`).

- ***impact_calculation_total***

This method receives a dictionary with the exchanges for the main process, a dictionary with the unit impacts for each sub-process, two integers (construction years and sub-processes), and a string (main process name). It validates that the impacts and exchanges align in length and that either both construction counts are zero or both non-zero, then splits impacts into infrastructure versus operational based on the number of the construction sub-process. It calls `calc.impact_calculation_total_no_contrs_ability`

which scales the sub-process unit impact with the exchange amounts, and stores the six returned arrays under `total_impact[name]`, `total_impact_contributions[name]`, `total_impact_infr[name]`, `total_impact_contributions_infr[name]`, `total_impact_op[name]`, `total_impact_contributions_op[name]`. It returns `None`, with all outputs as side-effects on the instance.

- ***sum_inventory_per_sub_sub_process***

This method takes a dictionary mapping each process key to a list of sparse inventory matrices (one per sub-sub-process), iterates over each list, sums each matrix row-wise, collects those arrays into a list, and stores that list under the same key in the `summed_unit_inventories` attribute. It returns `None`, with all results written as a side-effect to the instance attribute.

- ***sum_characterized_inventory_per_sub_sub_process***

This method operates as the `sum_inventory_per_sub_sub_process` function but for each impact category.

- ***emissions_calculation_in_time***

Does the same with the `calculation_static_lcia` function but for the emissions.

- ***characterized_inventory_calculation_in_time***

Does the same with the `calculation_static_lcia` function but for the characterized emissions.

- ***emissions_calculation_in_time_total***

Does the same with the `impact_calculation_total` function but for the emissions.

- ***characterized_inventory_calculation_in_time_total***

Does the same with the `impact_calculation_total` function but for the characterized emissions.

- ***calculate_total_impact_per_category_and_scenario***

This method takes the process name, queries the `self.total_impact` dictionary, passes the resulting array to `calc.calculate_total_impact_per_category_and_scenario(...)` function that adds the impact causes along the time steps and for each scenario, transposes the result, and stores it in the `self.total_impact_per_category_and_scenario[name]` attribute.

- ***calculate_total_fu_per_scenario***

Sums the total functional unit produced across the time steps in each scenario.

- ***per_fu_per_category_per_scenario***

Uses the total functional unit and the total impact caused to calculate the impact per functional unit across the scenarios.

- ***calculation_of_the_in_time_impact_evolution_absolute***

Calculates the cumulative and non-cumulative evolution of the impact caused.

- ***cum_calculation_of_the_in_time_impact_evolution_per_fu_actual***

Calculates the cumulative and non-cumulative evolution of the impact caused per functional unit produced.

- ***calculation_dynamic_LCIA_fcf***

Calculates the impact (for the main process and per environmental flow) calculated using FCFs

$$H = G * FCF$$

Where, G is the inventory matrix and FCF the FCF matrix.

The results are stored in attributes of the mpLCAer() instance, total_impact_fcf_per_flow, total_impact_fcf.

- ***calculation_dynamic_LCIA_icf***

Calculates the impact (for the main process) calculated using ICFs:

$$H_{j,\tau} = \sum_{t=0}^{\tau} (G_{j,t} * ICF_{j,\tau-t+1})$$

Where, G is the inventory matrix and ICF the ICF matrix. J is the environmental flow, t is the year and tau the year the emission was emitted.

The results are stored in attributes of the mpLCAer() instance, total_impact_icf_per_flow, total_impact_icf.

- ***calculate_total_impact_per_category_and_scenario_fcf***

Same as *calculate_total_impact_per_category_and_scenario* but takes as input the name of the impact category to be assessed.

- ***per_fu_per_category_per_scenario_fcf***

Same as *per_fu_per_category_per_scenario* but calculates the impact for the FCFs.

- ***calculation_of_the_in_time_impact_evolution_absolute_fcf***

Same as *calculation_of_the_in_time_impact_evolution_absolute* but takes as input the name of the impact category to be assessed.

- ***cum_calculation_of_the_in_time_impact_evolution_per_fu_actual_fcf***

Same as *calculation_of_the_in_time_impact_evolution_absolute* but takes as input the name of the impact category to be assessed.

lcpy.calculators.econ_calc

- *FastTeaCalculator*

A class that runs cython functions to quickly scale unit costs and revenues with the relevant exchange amounts for multidimensional calculations and to calculate several techno-economic KPIs. It provides a class that communicates the modelling of the user to cython functions and stores the results in its attributes which are then used to make the results analysis, storage, and visualization.

- ***cost_calculation***

This method takes a dictionary (mapping) of process keys to their exchange-cost arrays, a 1D array of discount rates (*discount_rate*), an optional scalar (*num*) to multiply costs, and a boolean (*infrastructure*); it calls *calc.cost_calculation_time_series* for each process, storing the returned per-process time-series array in *self.cost_per_process[key]* and the summed scenario×time 2D array in *self.cost[key]*. It returns None (all results are saved on the instance).

calc.cost_calculation_time_series: This function takes a list of 2D cost arrays (*exchange_costs*, one per process with shape [*scenarios*×*time_steps*]), a 1D array of discount rates (*discount_rates* of length *time_steps*), a scalar multiplier (*sign*), and a boolean *infrastructure*. It allocates a 3D array *result_groups* (*processes*×*scenarios*×*time_steps*) and a 2D *final_result* (*scenarios*×*time_steps*), then for each process, scenario, and time step either multiplies the cost by *sign* (if *infrastructure*=True) or by *discount_rates[t] * sign* (otherwise), storing into *result_groups*. Finally, it sums across processes into *final_result* and returns the tuple (*result_groups*, *final_result*).

- ***revenue_calculation***

Does the same job with *cost_calculation* but for the revenues.

- ***total_cost_calculation***

This method receives a discount rate array (`discount_rate`), a scenario name (`name`), a scalar multiplier (`num`), a flag to include construction discounting (`add_disc`), and a list of subprocesses to exclude (`excl_sub_processes`). It filters out excluded and infrastructure-related cost flows from `self.cost`, determines the appropriate construction discount flag, calls `calc.total_cost_per_time_step` with those flows, the discount rates, and discount parameters, and stores the returned non-cumulative and cumulative cost arrays—both overall and per process—into `self.non_cum_cost[name]`, `self.non_cum_cost_per_process[name]`, `self.cum_cost[name]`, and `self.cum_cost_per_process[name]`. All outputs are kept as instance attributes; the method returns `None`.

calc.total_cost_per_time_step: This function takes a list of 2D cost arrays (`cost_list`) for each process (shape: `scenarios*time_steps`), a 1D array of discount rates (`discount_rates`), a scalar sign multiplier (`sign`), and an integer index (`add_disc`) indicating which discount rate to apply (or zero for none). It allocates four NumPy arrays—non-cumulative sums (`scenarios*time_steps`), non-cumulative per-process sums (`scenarios*time_steps*processes`), cumulative sums, and cumulative per-process sums—then loops over processes, scenarios, and time steps to compute $-\text{cost} * \text{sign} * (\text{discount if } \text{add_disc} \neq 0)$ for the non-cumulative arrays, accumulates them into cumulative arrays, and returns the tuple (`non_cumulative_sum`, `non_cumulative_sum_per_process`, `cumulative_sum`, `cumulative_sum_per_process`).

- ***total_revenue_calculation***

Same with *total_cost_calculation* but for the revenues.

- ***total_cost_calculation_including_infrastructure***

Same with *total_cost_calculation* but includes the construction activities.

- ***npv_calculation***

This method uses the `self.cost` and `self.revenue` attributes and excludes some processes given to the function. It splits the remaining cost flows into operational and infrastructure lists, and then calls `calc.npv_calculation` with those lists, the instance's construction years, and the given discount rate. It stores the returned NPV, total costs, operational costs, and total revenue arrays under `self.npv[name]`, `self.total_costs[name]`, `self.operational_costs[name]`, and `self.total_revenue[name]`.

calc.npv_calculation: This function takes lists of 2D arrays—`processes_costs` (operational costs), `infrastructure_cost` (construction costs), and `processes_revenues` (revenues)—along with an integer `con_years` selecting which discount rate to apply for operations/revenues and a 1D array `discount_rates`. For each scenario `j`, it computes:

- Discounted operational cost by summing every process k and time step i using rate $r = \text{discount_rate}[\text{construction_years}]$.
- Discounted infrastructure cost by summing every construction process k and construction year i with rate r: $\text{discount_rate}[i]$.
- Discounted total revenue similarly.

It then forms total cost as the sum of operational and construction costs, and NPV as revenue plus total cost. The function returns four 1D arrays (length = number of scenarios: NPV, TotalCosts, OperationalCosts, TotalRevenue).

- ***npv_calculation_depreciation***

This method filters out any excluded subprocesses from `self.cost` and `self.revenue`, splits the remaining cost flows into operational and infrastructure groups, and then calls `calc.npv_calculation_depreciation` with those flows plus the instance's construction years, the provided `discount_rate`, `tax_rate`, `loan_percentage`, `loan_years`, `loan_rate`, and `depreciation_years`. It stores the returned arrays—net present value, total product costs, operational costs, total revenue, tax costs, and loan costs—into `self.npv[name]`, `self.total_product_costs[name]`, `self.operational_costs[name]`, `self.total_revenue[name]`, `self.tax_cost[name]`, and `self.loan_cost[name]`.

calc.npv_calculation_depreciation: This function takes lists of operational costs (`processes_costs`), revenues (`processes_revenues`), and infrastructure costs (`infrastructure_cost`), along with construction years index (`con_years`), discount rates (`discount_rates`), a tax rate (`tax_rate`), loan terms (`loan_percentage`, `loan_years`, `loan_rate`), and depreciation span (`depreciation_years`). It computes, for each scenario, (1) annual depreciation deductions, (2) loan amortization schedule and discounted payments, (3) before-tax cash flows and taxes, (4) discounted operational costs and revenues, (5) net product costs including infrastructure minus loans, and (6) the net present value (NPV). It returns six 1D arrays over scenarios:

- ***sum_sub_process***

Sums all costs/revenues (exchanges) in a provided list with exchanges.